αρχείο ή να τα διαβάζουμε από ένα αρχείο. Καταρχάς, θα πρέπει κάπως να αναπαριστούμε τα δεδομένα αυτά στη μνήμη του προγράμματός μας. Ο τυπικός τρόπος είναι ο σχεδιασμός κατάλληλης κλάσης. Έστω λοιπόν η κλάση Student, παράγωγη της Person.

```
import java.time.LocalDate;
import java.time.Month;
import java.util.Locale;

public class Student extends Person {

    private final double[] scores = new double[Subject.values().length];
    public final static int RECSIZE = 69;

    public Student(String id, String fName, String sName, LocalDate
birthday) {
        super(id, fName, sName, birthday);
    }

    public void setScore(Subject subject, double score) {
        scores[subject.ordinal()] = score;
    }

    public void setScores(double[] scores) {
        System.arraycopy(scores, 0, this.scores, 0, scores.length <
this.scores.length ? scores.length : this.scores.length);
    }

    public double getScore(Subject subject) {
        return scores[subject.ordinal()];
    }

    @Override
    public String toString() {
        return getId() + " " + getfName() + " " + getsName()
                + String.format(Locale.US, "%5.1f", getScore(Subject.MATH))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.HISTORY))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.LANGUAGE))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.CHEMISTRY))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.PHYSICS))
                + String.format(Locale.US, "%5.1f",
getScore(Subject.BIOLOGY))
                + " " + getBirthday();
    }

    public String toFixedWidth() {
        return String.format("%4s", getId())
                + String.format("%15s", getfName())
                + String.format("%15s", getsName())
                + String.format(Locale.US, "%4.1f", getScore(Subject.MATH))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.HISTORY))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.LANGUAGE))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.CHEMISTRY))
                + String.format(Locale.US, "%4.1f",
getScore(Subject.PHYSICS))
```

```
                + String.format(Locale.US, "%4.1f",
    getScore(Subject.BIOLOGY))
                + getBirthday();
        }
    }
```

Κώδικας 15.2 *Η κλάση Student.*

Κάθε αντικείμενο Student εκτός από τα χαρακτηριστικά που κληρονομεί από την Person έχει και έναν πίνακα scores στον οποίο αποθηκεύονται οι βαθμοί του. Οι βαθμοί ελέγχονται με τη βοήθεια του απαριθμήσιμου τύπου Subject.

```
public enum Subject {
     MATH, HISTORY, LANGUAGE, CHEMISTRY, PHYSICS, BIOLOGY;
}
```

Ο δημιουργός της κλάσης καλεί απλώς τον δημιουργό της γονικής, ενώ οι μέθοδοι setScores, setScore και getScore χρησιμεύουν ως αναγνώστες και ρυθμιστές για τους βαθμούς του μαθητή.

Αν υποθέσουμε τώρα πως θέλουμε να γράψουμε ή να διαβάσουμε τα δεδομένα μιας ομάδας μαθητών, έχουμε δύο ειδών τεχνικές στη διάθεσή μας.

1. Τα δεδομένα ενός μαθητή διαχωρίζονται μεταξύ τους με κατάλληλο χαρακτήρα. Στην περίπτωσή μας, μπορούμε να χρησιμοποιήσουμε ένα διάστημα. Ωστόσο, αν η κλάση Student διέθετε μια μεταβλητή, π.χ. name στην οποία εκχωρούνταν τιμές που περιλαμβάνουν διάστημα, π.χ. “John Black”, τότε είναι προφανές πως θα έπρεπε να χρησιμοποιήσουμε άλλο χαρακτήρα διαχωρισμού. Τα αρχεία που διαμορφώνονται σύμφωνα με αυτήν την τεχνική ονομάζονται οριοθετημένα αρχεία (delimited files).
2. Κάθε πεδίο από τα δεδομένα ενός μαθητή εγγράφεται σε προκαθορισμένο πλάτος. Για παράδειγμα, κάθε βαθμός μπορεί να εγγραφεί σε πλάτος 4 χαρακτήρων καθώς οι βαθμοί μας παρακολουθούνται με ακρίβεια 1 δεκαδικού και δεν υπερβαίνουν τις 2 ακέραιες θέσεις. Μία θέση επιπλέον χρειάζεται για το διαχωριστικό των δεκαδικών, π.χ. 19.5. Το όνομα μπορεί να εγγραφεί σε πλάτος 15 χαρακτήρων εφόσον εκτιμούμε ότι στα δεδομένα μας δεν υπάρχει όνομα που να υπερβαίνει τους 15 χαρακτήρες. Η τεχνική αυτή ονομάζεται οριοθέτηση σταθερού πλάτους (fixed width).

Η συνάρτηση toString της κλάσης Student επιστρέφει μια String αναπαράσταση του αντικειμένου Student όπου κάθε επιμέρους στοιχείο διαχωρίζεται από το προηγούμενο με ένα διάστημα. Επομένως, ένα αντικείμενο Student μπορεί να γραφεί σε ροή εξόδου σύμφωνα με την πρώτη τεχνική οριοθέτησης χωρίς καμιά περαιτέρω επεξεργασία.

Η συνάρτηση toFixedWidth επιστρέφει μια String αναπαράσταση του Student με βάση την οριοθέτηση σταθερού πλάτους.

Ο προσδιοριστής διαμόρφωσης που λαμβάνει η String.format υπακούει στους ίδιους κανόνες με αυτόν που παρουσιάστηκε στην ενότητα 4.2.1 για την printf.

Στη συνέχεια, παρουσιάζεται πρώτα ένα παράδειγμα που γράφει και διαβάζει στοιχεία μαθητών σε οριοθετημένο αρχείο και ένα δεύτερο παράδειγμα που γράφει και διαβάζει σε αρχεία σταθερού πλάτους.

```
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Scanner;

public class StudentsDelimited {

    static void createStudents(String fName) throws IOException {
```

```
        File file = new File(fName);
        BufferedWriter out = new BufferedWriter(new FileWriter(file));
        Student jim = new Student("113", "Jim", "Blue", LocalDate.of(2005,
Month.APRIL, 1));
        jim.setScores(new double[]{10, 11, 12, 13, 14, 15});
        Student jack = new Student("212", "Jack", "Green",
LocalDate.of(2005, Month.MAY, 1));
        jack.setScores(new double[]{11, 12, 13, 14, 15, 16});
        Student mary = new Student("111", "Mary", "Red", LocalDate.of(2005,
Month.JUNE, 1));
        mary.setScores(new double[]{12, 13, 14, 15, 16, 17});
        Student kely = new Student("201", "Kely", "Green",
LocalDate.of(2005, Month.JULY, 1));
        kely.setScores(new double[]{13, 14, 15, 16, 17, 18});
        out.write(jim + "\n");
        out.write(jack + "\n");
        out.write(mary + "\n");
        out.write(kely + "\n");

        out.close();
    }

    static ArrayList<Student> readStudents(String fName) throws
FileNotFoundException  {
        ArrayList<Student> rVal = new ArrayList<>();
        File input = new File(fName);
        Scanner scanner = new Scanner(input);
        while (scanner.hasNextLine()) {
            String st = scanner.nextLine();
            String[] stFields = st.split(" ");
            Student student = new Student(stFields[0], stFields[1],
stFields[2], LocalDate.parse(stFields[9]));
            student.setScore(Subject.MATH, Double.parseDouble(stFields[3]));
            student.setScore(Subject.HISTORY,
Double.parseDouble(stFields[4]));
            student.setScore(Subject.LANGUAGE,
Double.parseDouble(stFields[5]));
            student.setScore(Subject.CHEMISTRY,
Double.parseDouble(stFields[6]));
            student.setScore(Subject.PHYSICS,
Double.parseDouble(stFields[7]));
            student.setScore(Subject.BIOLOGY,
Double.parseDouble(stFields[8]));
            rVal.add(student);
        }
        scanner.close();
        return rVal;
    }

    public static void main(String[] args) throws IOException {
        String fName = "data" + File.separator + "studentsd.dat";
        createStudents(fName);
        ArrayList<Student> students = readStudents(fName);
        System.out.println(students);
    }
}
```

Κώδικας 15.3 *Ανάγνωση και εγγραφή σε οριοθετημένο αρχείο*

Στον κώδικα 15.3 η συνάρτηση createStudents δημιουργεί έναν αριθμό από αντικείμενα Students και τα αποθηκεύει σε αρχείο. Το αρχείο το διαχειριζόμαστε μέσω ενός FileWriter που είναι κλάση κατάλληλη για να

γράφουμε δεδομένα με τη μορφή χαρακτήρων. Επιπλέον χρησιμοποιείται ως περίβλημα ένα αντικείμενο BufferedWriter για επιτάχυνση της διαδικασίας εγγραφής. Κάθε μαθητής εισάγεται στο αρχείο ακολουθούμενος από τον χαρακτήρα newline. Μετά την εκτέλεση της createStudents, το αρχείο 'studentsd.dat' διαμορφώνεται ως ακολούθως:

```
113 Jim Blue 10.0 11.0 12.0 13.0 14.0 15.0 2005-04-01
212 Jack Green 11.0 12.0 13.0 14.0 15.0 16.0 2005-05-01
111 Mary Red 12.0 13.0 14.0 15.0 16.0 17.0 2005-06-01
201 Kely Green 13.0 14.0 15.0 16.0 17.0 18.0 2005-07-01
```

Η readStudents επιστρέφει ένα ArrayList από αντικείμενα Student που φορτώνει από το 'studentsd.dat'. Για να επιτύχει τη φόρτωση χρησιμοποιεί ένα αντικείμενο Scanner με το οποίο διαβάζει τις γραμμές του αρχείου μια προς μια. Η γραμμή εύκολα αποσυντίθεται στα επιμέρους στοιχεία της με την split. Στη συνέχεια, γνωρίζοντας τη γραμμογράφηση του αρχείου, μετατρέπει τα επιμέρους στοιχεία της γραμμής στους κατάλληλους τύπους και δημιουργεί το αντικείμενο Student που προσθέτει στο ArrayList που επιστρέφει. Τέλος, κλείνει τον Scanner. Στην main περιλαμβάνεται βασικός κώδικας ελέγχου.

```
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;
import java.util.Scanner;

public class StudentsToStdWidth {

    static void createStudents(String fName) throws IOException {
        File file = new File(fName);
        BufferedWriter out = new BufferedWriter(new FileWriter(file));
        Student jim = new Student("113", "Jim", "Blue", LocalDate.of(2005,
Month.MARCH, 1));
        jim.setScores(new double[]{10, 11, 12, 13, 14, 15});
        Student jack = new Student("212", "Jack", "Green",
LocalDate.of(2005, Month.MARCH, 2));
        jack.setScores(new double[]{11, 12, 13, 14, 15, 16});
        Student mary = new Student("911", "Mary", "Red", LocalDate.of(2005,
Month.MARCH, 3));
        mary.setScores(new double[]{12, 13, 14, 15, 16, 17});
        Student kely = new Student("201", "Kely", "Green",
LocalDate.of(2005, Month.MARCH, 4));
        kely.setScores(new double[]{13, 14, 15, 16, 17, 18});
        out.write(jim.toFixedWidth() + "\n");
        out.write(jack.toFixedWidth() + "\n");
        out.write(mary.toFixedWidth() + "\n");
        out.write(kely.toFixedWidth() + "\n");
        out.close();
    }

    public static Student toStudent(String st) {
        Student student = new Student(st.substring(0, 4).trim(),
                st.substring(4, 19).trim(),
                st.substring(19, 34).trim(),
                LocalDate.parse(st.substring(58))
        );
```

```
            student.setScore(Subject.MATH, Double.parseDouble(st.substring(34,
38).trim()));
            student.setScore(Subject.HISTORY,
Double.parseDouble(st.substring(38, 42).trim()));
            student.setScore(Subject.LANGUAGE,
Double.parseDouble(st.substring(42, 46).trim()));
            student.setScore(Subject.CHEMISTRY,
Double.parseDouble(st.substring(46, 50).trim()));
            student.setScore(Subject.PHYSICS,
Double.parseDouble(st.substring(50, 54).trim()));
            student.setScore(Subject.BIOLOGY,
Double.parseDouble(st.substring(54, 58).trim()));
            return student;
        }

        static ArrayList<Student> readStudents(String fName) throws
FileNotFoundException {
            ArrayList<Student> rVal = new ArrayList<>();
            File input = new File(fName);
            Scanner scanner = new Scanner(input);
            while (scanner.hasNextLine()) {
                String st = scanner.nextLine();
                Student student = toStudent(st);
                rVal.add(student);
            }
            scanner.close();
            return rVal;
        }

        public static void main(String[] args) throws IOException {
            createStudents("data" + File.separator + "students.dat");
            ArrayList<Student> students = readStudents("data" + File.separator
+ "students.dat");
            System.out.println(students);
        }
}
```

Κώδικας 15.4 *Ανάγνωση και εγγραφή σε αρχείο σταθερού πλάτους*

Στον κώδικα 15.4 επιδεικνύεται η ανάγνωση και εγγραφή σε αρχείο σταθερού πλάτους. Μετά από εκτίμηση του μέγιστου πιθανού πλάτους των δεδομένων ενός Student σχεδιάστηκε η ακόλουθη γραμμογράφηση του αρχείου. Το id τοποθετείται σε πλάτος 4 χαρακτήρων, το όνομα σε πλάτος 15, το επώνυμο σε πλάτος 15, οι βαθμοί σε πλάτος 4 με ακρίβεια 1 δεκαδικού. Τέλος, η ημερομηνία γέννησης γράφεται και διαβάζεται με την εξ ορισμού διαμόρφωσή της που καταλαμβάνει πλάτος 10 (yyyy-mm-dd). Η γραμμογράφηση αυτή επιστρέφεται από τη συνάρτηση toStandardWidth της κλάσης Student. Επομένως, η createStudents, το μόνο που κάνει είναι να δημιουργεί μια σειρά από αντικείμενα Student και να ενημερώνει το αρχείο students.dat. Η εγγραφή στο αρχείο γίνεται μέσω ενός FileWriter που είναι η κατάλληλη κλάση για εγγραφή χαρακτήρων σε αρχεία. Για επιτάχυνση της διαδικασίας εγγραφής χρησιμοποιείται ένα BufferedWriter ως περίβλημα του αντικειμένου FileWriter.
Μετά την εκτέλεση της createStudent, το αρχείο students.dat έχει ως εξής:

```
113            Jim            Blue10.011.012.013.014.015.02005-03-01
212           Jack           Green11.012.013.014.015.016.02005-03-02
911           Mary             Red12.013.014.015.016.017.02005-03-03
201           Kely           Green13.014.015.016.017.018.02005-03-04
```

Στη συνέχεια, η readStudents διαβάζει μία προς μία τις γραμμές του αρχείου και τις μετατρέπει σε αντικείμενα Student με τη βοήθεια της toStudent. Είναι προφανές πως η ανάγνωση τόσο οριοθετημένων αρχείων όσο και αρχείων σταθερού πλάτους προϋποθέτει τη γνώση της δομής τους. Στην main παρατίθεται βασικός κώδικας ελέγχου.

15.3 Ανάγνωση και εγγραφή σε αρχεία ψηφιολέξεων

Είδαμε πως μπορούμε να γράφουμε και να διαβάζουμε χαρακτήρες από ένα αρχείο. Μερικές φορές όμως τα αρχεία που θέλουμε να επεξεργαστούμε δεν είναι αρχεία κειμένου ώστε να περιέχουν χαρακτήρες. Για παράδειγμα, έστω ότι θέλουμε να αντιγράψουμε ένα αρχείο εικόνας. Σε αυτήν την περίπτωση θα πρέπει να διασυνδεθούμε μέσω ροής ψηφιολέξεων.

```
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;

public class BinaryData {

    private static final String INPUT = "data" + File.separator +
"sunset.jpg";
    private static final String OUTPUT = "data" + File.separator +
"sunsetcopy.jpg";
    private static final String OUTPUTF = "data" + File.separator +
"sunsetcopyf.jpg";

    public static void copyBinary() throws FileNotFoundException,
IOException {
        FileInputStream in = new FileInputStream(INPUT);
        FileOutputStream out = new FileOutputStream(OUTPUT);
        int data;
        while (true) {
            data = in.read();
            if (data == -1) {
                break;
            } else {
                out.write(data);
            }
        }
        in.close();
        out.close();
    }

    public static void copyBinaryFast() throws FileNotFoundException,
IOException {
        BufferedInputStream in = new BufferedInputStream(new
FileInputStream(INPUT));
        BufferedOutputStream out = new BufferedOutputStream(new
FileOutputStream(OUTPUTF));
        int data;
        while (true) {
            data = in.read();
            if (data == -1) {
                break;
            } else {
                out.write(data);
            }
        }
        in.close();
        out.close();
    }

    public static void main(String[] args) throws IOException {
```

```
            copyBinary();
            copyBinaryFast();
        }
    }
```

Κώδικας 15.5 *Ανάγνωση και εγγραφή σε αρχεία ψηφιολέξεων*

Η κλάση BinaryData περιλαμβάνει δύο στατικές συναρτήσεις, την copyBinary και την copyBinaryFast. Και οι δύο αντιγράφουν την εικόνα sunset.jpg σε νέο αρχείο. Η copyBinary χρησιμοποιεί ένα αντικείμενο FileInputStream για ανάγνωση ψηφιολέξεων και ένα αντικείμενο FileOutputStream για εγγραφή ψηφιολέξεων. Η copyBinaryFast χρησιμοποιεί επιπλέον τα περιβλήματα BufferedInputStream και BufferedOutputStream. Δεδομένου ότι και οι δύο επιτελούν την ίδια εργασία, είναι εύκολη η εκτίμηση της επιτάχυνσης που προσφέρουν οι ροές προσωρινής μνήμης στις εφαρμογές. Τρέξτε τις δύο συναρτήσεις χωριστά και εκτιμήστε τη χρονική διαφορά στην εκτέλεσή τους.

Μετά το άνοιγμα της ροής, μέσα στην επαναληπτική διαδικασία, οι συναρτήσεις διαβάζουν μια προς μια τις ψηφιολέξεις. Η read διαβάζει την τρέχουσα ψηφιολέξη την οποία και επιστρέφει, οπότε στη συνέχεια η write την γράφει στο αρχείο εξόδου. Αν ωστόσο η read συναντήσει τέλος αρχείου, τότε επιστρέφει -1, οπότε διακόπτεται η επαναληπτική διαδικασία. Τέλος, οι συναρτήσεις κλείνουν τις ροές.

15.4 Αρχεία τυχαίας προσπέλασης

Ένας άλλος πολύ σημαντικός τρόπος διαχείρισης των αρχείων είναι η διαχείριση με τυχαία προσπέλαση (Random access). Η τυχαία προσπέλαση μας δίνει τη δυνατότητα να διαβάσουμε ή να γράψουμε άμεσα σε οποιοδήποτε σημείο του αρχείου θέλουμε. Ο χρόνος που απαιτείται για την προσπέλαση είναι ίδιος είτε προσπελαύνουμε την αρχή του αρχείου είτε οποιοδήποτε άλλο σημείο.

Η τυχαία προσπέλαση αξιοποιείται συνήθως σε δομημένα αρχεία. Ας πάρουμε για παράδειγμα το αρχείο student.dat που δημιουργήθηκε από τον κώδικα 15.4. Σε αυτό τα στοιχεία κάθε μαθητή καταλαμβάνουν μία γραμμή και συνιστούν μια εγγραφή (record). Όλες οι εγγραφές στο αρχείο έχουν το ίδιο πλάτος. Καθώς η συνάρτηση που παράγει αυτές τις εγγραφές, η toFixedWidth βρίσκεται στην κλάση Student, εκεί έχουμε τοποθετήσει και την πληροφορία σχετικά με το μήκος της εγγραφής. Πρόκειται για τη στατική μεταβλητή RECSIZE. Τα αρχεία τυχαίας προσπέλασης υποστηρίζουν τον δείκτη αρχείου (file pointer). Οι λειτουργίες ανάγνωσης και εγγραφής αρχίζουν από το σημείο που υποδεικνύει ο δείκτης αρχείου. Αν για παράδειγμα, θέλουμε να διαβάσουμε την πρώτη εγγραφή του αρχείου, ο δείκτης πρέπει να δείχνει στην ψηφιολέξη 0, αν θέλουμε να διαβάσουμε τη δεύτερη εγγραφή, ο δείκτης πρέπει να δείχνει στην ψηφιολέξη RECSIZE, αν θέλουμε την τρίτη εγγραφή, ο δείκτης πρέπει να δείχνει στην ψηφιολέξη 2*RECSIZE, κοκ.

Η κλάση που μας εξυπηρετεί για τυχαία προσπέλαση είναι η RandomAccessFile. Ο δείκτης του αρχείου τοποθετείται με τη μέδοθο seek. Επίσης, μπορούμε να μάθουμε πού δείχνει ο δείκτης με τη μέθοδο getFilePointer.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.RandomAccessFile;
import java.time.LocalDate;
import java.time.Month;

public class RandomAccess {

    public static Student read(RandomAccessFile f, int recNo) throws
IOException {
        int records = (int) (f.length() / Student.RECSIZE);
        if (recNo < 0 || recNo >= records) {
            throw new RuntimeException("reading past of file end");
        }
        long pos = recNo * Student.RECSIZE;
        f.seek(pos);
        String s = f.readLine();
```

```
            Student student = StudentsToStdWidth.toStudent(s);
            return student;
        }

        public static void update(RandomAccessFile f, int recNo, Student
    student) throws IOException {
            int records = (int) (f.length() / Student.RECSIZE);
            if (recNo < 0 || recNo >= records) {
                throw new RuntimeException("updating past of file end");
            }
            long pos = recNo * Student.RECSIZE;
            f.seek(pos);
            f.writeBytes(student.toFixedWidth());
        }

        public static void append(RandomAccessFile f, Student student) throws
    IOException {
            f.seek(f.length());
            f.writeBytes(student.toFixedWidth()+"\n");
        }

        public static void main(String[] args) throws FileNotFoundException,
    IOException {
            RandomAccessFile f = new
    RandomAccessFile("data"+File.separator+"students.dat", "rw");
            Student student = read(f, 3);
            System.out.println(student);
            student.setfName("Mike");
            student.setsName("Updater");
            update(f, 3, student);
            student=new Student("999", "Jane", "Appender",
    LocalDate.of(1999,Month.MARCH,20));
            student.setScores(new double[]{14,15,16,17,18,19});
            append(f,student);
            append(f,student);
            f.close();
        }
    }
```

Κώδικας 15.6 *Ανάγνωση και εγγραφή σε αρχεία με τυχαία προσπέλαση*

Η συνάρτηση read στον κώδικα 15.6 λαμβάνει ως παραμέτρους ένα αντικείμενο RandomAccessFile και έναν ακέραιο. Ο ακέραιος αναπαριστά τον αύξοντα αριθμό της εγγραφής μέσα στο αρχείο. Η συνάρτηση length της RandomAccessFile επιστρέφει το συνολικό μήκος του αρχείου σε ψηφιολέξεις. Το μήκος αυτό διαιρούμενο δια του μήκους της εγγραφής μας δίνει τον αριθμό των εγγραφών στο αρχείο. Έτσι, γίνεται έλεγχος μήπως ο αριθμός εγγραφής που ζητήθηκε από την read είναι εκτός των ορίων του αρχείου. Αν όχι, ο δείκτης τοποθετείται στη θέση recNo * Student.RECSIZE. Μετά την τοποθέτηση του δείκτη, διαβάζουμε μια εγγραφή και δημιουργούμε το αντικείμενο Student με την toStudent της StudentsToStdWidth.

Η συνάρτηση append μετακινεί τον δείκτη στο τέλος του αρχείου όπου προσθέτει μια νέα εγγραφή με τη βοήθεια της writeBytes και της toFixedWidth.

Η συνάρτηση update λαμβάνει ως παραμέτρους το αρχείο, τον αριθμό εγγραφής και ένα αντικείμενο Student. Αντικαθιστά τα στοιχεία της εγγραφής που βρίσκεται στη θέση recNo*Student.RECSIZE με τα στοιχεία του αντικειμένου Student. Προσέξτε πως στην writeBytes δεν προστίθεται χαρακτήρας newline όπως στην αντίστοιχη writeBytes της append, καθώς ο χαρακτήρας υπάρχει ήδη στο αρχείο.

15.5 Σειριοποίηση

Ας υποθέσουμε πως σε μια εφαρμογή-παιχνίδι θέλουμε να δώσουμε τη δυνατότητα στον χρήστη να διακόπτει το παιχνίδι και να το συνεχίζει ακριβώς από το σημείο στο οποίο διέκοψε. Στην περίπτωση αυτή θα μας ήταν

χρήσιμο να αποθηκεύσουμε σε αρχείο τα αντικείμενα της εφαρμογής όπως αυτά είχαν διαμορφωθεί στη μνήμη κατά τη στιγμή της διακοπής έτσι ώστε να μπορούμε να τα φορτώσουμε από το αρχείο στη μνήμη όποτε χρειαστεί. Η διαδικασία με την οποία επιτυγχάνουμε αποθήκευση και φόρτωση αντικειμένων μεταξύ μνήμης και αρχείου ονομάζεται σειριοποίηση (serialization).

Θα δούμε τη σειριοποίηση στην πράξη με ένα παράδειγμα το οποίο σώζει ένα ArrayList από αντικείμενα κλάσης που αναπαριστά αυτοκίνητα και στη συνέχεια το φορτώνει και το εμφανίζει. Θα χρειαστούμε καταρχάς μια κλάση που αναπαριστά τα αυτοκίνητα.

```
import java.io.Serializable;
import java.util.Objects;

public class Car implements Serializable {
    private String brand;
    private int maximumSpeed;

    public Car(String brand, int maximumSpeed) {
        this.brand = brand;
        this.maximumSpeed = maximumSpeed;
    }

    public int getMaximumSpeed() {
        return maximumSpeed;
    }

    public void setMaximumSpeed(int maximumSpeed) {
        this.maximumSpeed = maximumSpeed;
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }

    @Override
    public String toString() {
        return "Car{" + "brand=" + brand + ", maximumSpeed=" + maximumSpeed
+ '}';
    }

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 67 * hash + Objects.hashCode(this.brand);
        hash = 67 * hash + this.maximumSpeed;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
```

```
            }
            final Car other = (Car) obj;
            if (this.maximumSpeed != other.maximumSpeed) {
                return false;
            }
            return Objects.equals(this.brand, other.brand);
        }
    }
```

Κώδικας 15.7 *Η κλάση Car*

Η κλάση Car στον κώδικα 15.7 είναι μια απλή κλάση που περιλαμβάνει τις αναγκαίες συναρτήσεις. Εκείνο που έχει παραπάνω σε σχέση με τις κλάσεις που έχουμε υλοποιήσει μέχρι τώρα είναι πως υλοποιεί την προκαθορισμένη διεπαφή Serializable. Η διεπαφή Serializable δεν ορίζει καμία αφηρημένη μέθοδο. Πρόκειται για διεπαφή-δείκτη (marker interface). Η υλοποίησή της από οποιαδήποτε κλάση απαιτεί μόνο τη δήλωσή της στη λίστα των διεπαφών που υλοποιεί η κλάση. Ωστόσο είναι απαραίτητη ώστε να επιτρέπεται η σειριοποίηση μιας κλάσης.

Ας προχωρήσουμε τώρα στην εφαρμογή που αποθηκεύει το ArrayList από αντικείμενα Car.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;

public class Serialization {

    static ArrayList<Car> createCars() {
        ArrayList<Car> store = new ArrayList<>();
        store.add(new Car("Toyota", 220));
        store.add(new Car("Ford", 230));
        store.add(new Car("Peugeot", 240));
        return store;
    }

    static void saveCars() throws IOException {
        ArrayList<Car> cars = createCars();
        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("data\\cars.dat"));
        out.writeObject(cars);
        out.close();
    }

    static ArrayList<Car> loadCars() throws IOException,
ClassNotFoundException {
        ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("data\\cars.dat"));
        ArrayList<Car> loaded = (ArrayList<Car>) in.readObject();//essential
unsafe operation
        in.close();
        return loaded;
    }

    public static void main(String[] args) throws IOException,
ClassNotFoundException {
        saveCars();
        ArrayList<Car> store = loadCars();
        System.out.println(store);
    }
}
```

Κώδικας 15.8 *Σειριοποίηση ενός ArrayList<Car>*

Στον κώδικα 15.8 η createCars δημιουργεί το ArrayList από αντικείμενα Car. Η saveCars ανοίγει ροή τύπου ObjectOutputStream που λειτουργεί ως περίβλημα σε FileOutputStream. Με την writeObject πραγματοποιείται η εγγραφή. Προσέξτε στην τεκμηρίωση της ArrayList πως η κλάση υλοποιεί τη διεπαφή Serializable, διαφορετικά η λειτουργία θα αποτύγχανε. Αντίστοιχα απλή είναι και η διαδικασία ανάγνωσης του αντικειμένου που γίνεται από την loadCars με χρήση ενός ObjectInputStream και ενός FileInputStream. Προσέξτε πως η readObject της ObjectInputStream επιστρέφει Object και άρα είναι απαραίτητη η μετατροπή του σε ArrayList<Car>.

15.6 Χρήσιμες λειτουργίες

Η κλάση File παρέχει μια σειρά από χρήσιμες λειτουργίες. Ένα αντικείμενο της κλάσης File αναπαριστά το όνομα της διαδρομής ενός αρχείου ή φακέλου [2]. Τα λειτουργικά συστήματα ονομάζουν τις διαδρομές των αρχείων με βάση τα ιδιαίτερα χαρακτηριστικά τους. Αυτά όμως διαφέρουν από λειτουργικό σε λειτουργικό. Καθώς η Java είναι διαπλατφορμική γλώσσα, χρειάζεται έναν τρόπο για να αναπαριστά τις ονομασίες των διαδρομών αρχείων και φακέλων που είναι ανεξάρτητος από το λειτουργικό σύστημα. Αυτόν τον τρόπο παρέχει η κλάση File.

Οι ονομασίες των διαδρομών αρχείων και φακέλων είναι δύο τύπων, οι σχετικές (relative) και οι πλήρεις (absolute). Η πλήρης διαδρομή επαρκεί για τον εντοπισμό του αρχείου που δηλώνει. Αντίθετα, μια σχετική διαδρομή πρέπει να συνδυαστεί με κάποια άλλη προκειμένου να εντοπιστεί το αρχείο που δηλώνει. Οι κλάσεις του πακέτου java.io επιλύουν τις σχετικές διαδρομές με βάση την ιδιότητα user.dir του συστήματος. Στην περίπτωσή μας, εφόσον δουλεύουμε στο NetBean, η διαδρομή που συνδυάζεται με την ιδιότητα user.dir αντιστοιχεί στον φάκελο του NetBeans project στο οποίο εργαζόμαστε.

Ας δούμε τώρα μια σειρά από χρήσιμες συναρτήσεις:

```
public static String getProperty(String key)
```

Η συνάρτηση getProperty της κλάσης System όταν κληθεί με παράμετρο “user.dir” επιστρέφει τον τρέχοντα φάκελο εργασίας.
Οι υπόλοιπες συναρτήσεις ανήκουν στην κλάση File.

```
boolean canExecute()
```

Ελέγχει αν η εφαρμογή μπορεί να τρέξει αυτό το αρχείο, δηλαδή το αρχείο που αναπαρίσταται από το αντικείμενο File που καλεί την canExecute.

```
boolean canRead()
```

Ελέγχει αν η εφαρμογή μπορεί να διαβάσει αυτό το αρχείο.

```
boolean canWrite()
```

Ελέγχει αν η εφαρμογή μπορεί να γράψει σε αυτό το αρχείο

```
boolean createNewFile()
```

Δημιουργεί ένα αρχείο με αυτό το όνομα εφόσον δεν υπάρχει ήδη.

```
boolean delete()
```

Διαγράφει το αρχείο ή τον φάκελο με αυτό το όνομα.

```
String getAbsolutePath()
```

Επιστρέφει την πλήρη διαδρομή αυτού του αρχείου.

```
String getName()
```

Επιστρέφει το όνομα αυτού του αρχείου.

```
boolean isDirectory()
```

Ελέγχει αν αυτό το αρχείο είναι φάκελος.

```
boolean isHidden()
```
Ελέγχει αν αυτό το αρχείο είναι κρυφό.

```
long lastModified()
```
Επιστρέφει τον χρόνο σε milisecond από την τελευταία τροποποίηση αυτού του αρχείου.

```
long length()
```
Επιστρέφει το μέγεθος σε bytes αυτού του αρχείου.

```
File[] listFiles()
```
Επιστρέφει έναν πίνακα από αρχεία που περιέχονται σε αυτόν τον φάκελο.

```
boolean mkdir()
```
Δημιουργεί έναν φάκελο με αυτό το όνομα.

15.7 Λυμένες Ασκήσεις

15.7.1 Περιοδικός Πίνακας

Σε ένα οριοθετημένο αρχείο κειμένου περιέχονται **όλα** τα χημικά στοιχεία του περιοδικού πίνακα με τα βασικά τους χαρακτηριστικά όπως φαίνεται στο ακόλουθο παράδειγμα:

```
H 1 1 s 1
Li 1 2 s 3
Na 1 3 s 11
Be 2 2 s 4
Mg 2 3 s 12
Sc 3 4 d 21
Y 3 5 d 39
He 18 1 p 2
La 3 6 f 57
```

Η γραμμογράφηση του αρχείου έχει ως εξής:

```
Σύμβολο Ομάδα Περίοδος Τομέας Ατομικός_Αριθμός
```

Προσθέστε στην κλάση TableOfChemichalElements συνάρτηση

```
public static TableOfChemichalElements getPeriodTable()
```

που επιστρέφει ένα αντικείμενο TableOfChemichalElements που περιλαμβάνει τα στοιχεία του αρχείου και επομένως αναπαριστά τον περιοδικό πίνακα. Φροντίστε ώστε η getPeriodTable να διαβάζει το αρχείο μόνο μία φορά κατά την τρέχουσα εκτέλεση του προγράμματος. Μετά την ανάγνωση να διατηρεί την επιστρεφόμενη τιμή στη μνήμη από όπου την επιστρέφει με κάθε κλήση της.

Λύση
Καταρχάς θα πρέπει να προσθέσουμε δύο στατικά μέλη στην κλάση.

```
private final static File F = new File("data" + File.separator
+ "ChemicalElements.dat");
private static TableOfChemichalElements periodTable = null;
```

Το F αφορά το αρχείο το οποίο περιέχει τα χημικά στοιχεία. Η getPeriodTable δεν θα φορτώνει δεδομένα από ένα οποιοδήποτε αρχείο αλλά από ένα συγκεκριμένο που περιέχει το σύνολο των χημικών στοιχείων. Με αυτήν την έννοια, το αρχείο δεν σχετίζεται με συγκεκριμένο στιγμιότυπο της TableOfChemichalElements, οπότε το F είναι static. Επίσης static είναι το periodTable που είναι ο πίνακας που επιστρέφει η getPeriodTable. Το periodTable αρχικοποιείται στην τιμή null. Μετά την πρώτη εκτέλεση της getPeriodTable αποτελεί αναφορά στο στιγμιότυπο TableOfChemichalElements που επιστρέφει η getPeriodTable.
Στη συνέχεια, αναπτύσσουμε τρεις εναλλακτικές λύσεις και συζητάμε τα πλεονεκτήματα και μειονεκτήματά τους.

```
     public static TableOfChemichalElements getPeriodTable() throws
FileNotFoundException {
        if (periodTable == null) {
            periodTable = new TableOfChemichalElements();
            Scanner scanner = new Scanner(F);
            while (scanner.hasNextLine()) {
                String line = scanner.nextLine();
                String[] elements = line.split(" ");
                ChemicalElement cE = new
ChemicalElement(Integer.parseInt(elements[1]), Integer.parseInt(elements[2]),
elements[3].charAt(0), Integer.parseInt(elements[4]), elements[0]);
                periodTable.add(cE);
            }
            scanner.close();
        }
        return periodTable;
    }
```

Κώδικας 15.9 *Η συνάρτηση getPeriodTable*

Στον κώδικα 15.9 η συνάρτηση getPeriodTable ελέγχει τη στατική μεταβλητή periodTable και αν την βρει null φορτώνει το αρχείο με τα χημικά στοιχεία στη μνήμη. Στη συνέχεια επιστρέφει την periodTable. Πράγματι με αυτήν την προσέγγιση θα επιστρέψει στον χρήστη της TableOfChemichalElements έναν πίνακα με χημικά στοιχεία. Έχει ενδιαφέρον όμως τι μπορεί να συμβεί από κει και πέρα.

```
     public static void main(String[] args) throws IOException {
        TableOfChemichalElements pT=TableOfChemichalElements.getPeriodTable();
        ChemicalElement li = new ChemicalElement(1, 2, 's', 3, "Pi");
        pT.add(li);
        System.out.println(TableOfChemichalElements.getPeriodTable());

    }
```

Κώδικας 15.10 *Αλλοίωση του πίνακα περιοδικών στοιχείων από την εφαρμογή*

Η getPeriodTable επιστρέφει την αναφορά στον TableOfChemicalElements.periodTable. Ο κώδικας 15.10 κρατάει αυτήν την αναφορά στην τοπική μεταβλητή pT. Επομένως, οι αναφορές TableOfChemicalElements.periodTable και pT βλέπουν στον ίδιο πίνακα στη μνήμη. Από τη στιγμή που θα γίνει μια μεταβολή στοιχείων του πίνακα από την εφαρμογή, π.χ. στον κώδικα 15.10 προστίθεται ένα στοιχείο, κάθε κλήση της getPeriodTable θα επιστρέφει όχι τον αρχικό πίνακα, αυτόν που φορτώθηκε από το αρχείο, αλλά τον πίνακα όπως μεταβλήθηκε από την εφαρμογή. Πρόκειται όμως για τον περιοδικό πίνακα των χημικών στοιχείων που είναι συγκεκριμένος και θα πρέπει πάντα να επιστρέφεται αμετάβλητος. Επομένως, η προσέγγιση αυτή παρουσιάζει σοβαρή αδυναμία.

Η λύση στο πρόβλημα αυτό είναι η getPeriodTable να δημιουργεί έναν νέο πίνακα, να τον ενημερώνει κατάλληλα και να τον επιστρέφει στον χρήστη της. Έτσι κάθε μεταβολή του πίνακα από την εφαρμογή δεν θα επηρεάζει τον πίνακα που θα επιστρέφει η getPeriodTable. Για να το πετύχουμε αυτό, θα πρέπει καταρχάς να προσθέσουμε δύο νέους δημιουργούς στην TableOfChemicalElements.

```
    public TableOfChemichalElements() {}
```

```
public TableOfChemichalElements(TableOfChemichalElements source) {
    Iterator<ChemicalElement> it = source.store.iterator();
    while (it.hasNext()) {
        store.add(it.next());
    }
}
```

Κώδικας 15.11 *Δύο δημιουργοί της κλάσης TableOfChemicalElements.*

Ο δημιουργός που λαμβάνει ως παράμετρο TableOfChemicalElements παράγει ένα αντίγραφο της παραμέτρου του. Προσέξτε πως τα χημικά στοιχεία στο αντίγραφο είναι τα ίδια με τα χημικά στοιχεία της παραμέτρου και όχι αντίγραφά τους. Αυτό είναι σωστό καθώς τα χημικά στοιχεία δεν διαθέτουν ρυθμιστές και επιπλέον όλες οι μεταβλητές στιγμιότυπου είναι σταθερές. Επομένως, ένα χημικό στοιχείο από τη στιγμή που θα δημιουργηθεί δεν μπορεί να μεταβληθεί και άρα δεν υπάρχει λόγος να παράγουμε αντίγραφο.

Ο δημιουργός χωρίς παραμέτρους είναι απαραίτητος για να τρέχουν οι κώδικες που μέχρι τώρα χρησιμοποιούσαν τον εξ ορισμού δημιουργό.

Έχοντας τους δύο δημιουργούς στη διάθεσή μας αρκεί να μετατρέψουμε την τελευταία γραμμή της getPeriodTable ώστε, αντί να επιστρέφει τη στατική periodTable, να επιστρέφει ένα αντίγραφό της όπως δείχνει ο κώδικας που ακολουθεί:

```
return new TableOfChemichalElements(periodTable);
```

Μία εναλακτική προσέγγιση βασίζεται στην αξιοποίηση της διεπαφής Cloneable. Όσες κλάσεις υλοποιούν τη διεπαφή Cloneable τους επιτρέπεται να καλέσουν τη συνάρτηση clone() που κληρονομούν από την Object. Εδώ το σχήμα φαίνεται λίγο περίεργο καταρχάς, αλλά έχει την εξήγησή του. Η συνάρτηση clone() που επιστρέφει ένα αντίγραφο του αντικειμένου που την καλεί τοποθετήθηκε στην κλάση Object σε αρχική έκδοση της Java. Επομένως, όλες οι κλάσεις την κληρονομούν. Σύντομα όμως έγινε αντιληπτό πως κάποιες κλάσεις δεν θα πρέπει να δίνουν τη δυνατότητα παραγωγής αντιγράφων των αντικειμένων τους. Προκειμένου να μην βγει η clone από την Object ώστε να διατηρηθεί η συμβατότητα με προηγούμενες εκδόσεις, οι σχεδιαστές της Java αποφάσισαν να ορίσουν τη διεπαφή Cloneable και να επιτρέψουν τη χρήση της clone στις κλάσεις που υλοποιούν την Cloneable. Επομένως, ο λόγος της περίεργης απαίτησης υλοποίησης μιας διεπαφής προκειμένου να υλοποιηθεί μια μέθοδος που κληρονομείται κρύβεται πίσω από μια αρχικά λανθασμένη σχεδιαστική επιλογή και μια εκ των υστέρων ευρετική λύση. Ας σημειωθεί πως οι κλάσεις που επιχειρούν να καλέσουν την clone χωρίς να έχουν υλοποιήσει τη διεπαφή clone παράγουν εξαίρεση τύπου CloneNotSupportedException.

Για την υλοποίηση αυτής της εναλακτικής θα πρέπει καταρχάς να δηλώσουμε στην επικεφαλίδα της TableOfChemicalElements ότι υλοποιεί τη διεπαφή Cloneable, όπως φαίνεται αμέσως παρακάτω:

```
public class TableOfChemichalElements implements Cloneable {…}
```

Στη συνέχεια πρέπει να επανορίσουμε τη συνάρτηση clone στην TableOfChemichalElements.

```
@Override
public Object clone() throws CloneNotSupportedException {
    TableOfChemichalElements rVal=new TableOfChemichalElements();
    Iterator<ChemicalElement> it = store.iterator();
    while (it.hasNext()) {
        rVal.add(it.next());
    }
    return rVal;
}
```

Κώδικας 15.12 *Η συνάρτηση clone της κλάσης TableOfChemicalElements.*

Προσέξτε πως η clone παράγει CloneNotSupportedException και επιστρέφει Object. Στη συνέχεια θα τροποποιήσουμε την getPeriodTable ως εξής:

```
    public static TableOfChemichalElements getPeriodTable() throws
FileNotFoundException, CloneNotSupportedException {
        if (periodTable == null) {
            periodTable = new TableOfChemichalElements();
            Scanner scanner = new Scanner(F);
            while (scanner.hasNextLine()) {
                String line = scanner.nextLine();
                String[] elements = line.split(" ");
                ChemicalElement cE = new
ChemicalElement(Integer.parseInt(elements[1]), Integer.parseInt(elements[2]),
elements[3].charAt(0),
                        Integer.parseInt(elements[4]), elements[0]);
                periodTable.add(cE);
            }
            scanner.close();
        }
        return (TableOfChemichalElements)periodTable.clone();
    }
```

Κώδικας 15.13 *Η συνάρτηση getPeriodTable βασισμένη στην clone.*

Στον κώδικα 15.13 η επιστροφή της συνάρτησης βασίζεται στην clone. Καθώς η clone επιστρέφει αντικείμενο τύπου Object, πρέπει υποχρεωτικά να προβούμε σε μετατροπή τύπου. Επιπλέον, στη συνάρτηση έχει δηλωθεί η πιθανή παραγωγή της CloneNotSupportedException.

Ανακεφαλαιώνοντας επισημαίνουμε πως η πρώτη λύση παρουσιάζει σοβαρή αδυναμία που μπορεί να οδηγήσει σε λάθη τον κώδικα μιας εφαρμογής. Αντίθετα, η δεύτερη και τρίτη λύση προστατεύει τον περιοδικό πίνακα. Οι χρήστες της κλάσης μπορούν να κάνουν μεταβολές στον πίνακα που θα λάβουν από την getPeriodTable αλλά η τιμή επιστροφής της μένει αμετάβλητη. Στην πράξη η δεύτερη και η τρίτη λύση είναι σε γενικές γραμμές ισοδύναμες.

15.7.2 Κατάλογοι αρχείων

Να υλοποιηθεί συνάρτηση public static void dirRecurse(File root) που λαμβάνει ως παράμετρο και εμφανίζει τα περιεχόμενα του root. Αν το root αντιστοιχεί σε κατάλογο αρχείων, εμφανίζει στοιχεία για κάθε αρχείο του καταλόγου και το ίδιο κάνει για κάθε υποκατάλογο του root. Αν το root αντιστοιχεί σε απλό αρχείο, εμφανίζει πληροφορίες για το αρχείο. Για κάθε κατάλογο από το root και κάτω θα πρέπει να εμφανίζεται πρώτα η ονομασία του και να ακολουθεί λίστα με πληροφορίες για κάθε αρχείο του καταλόγου σύμφωνα με το υπόδειγμα που ακολουθεί:

```
   Directory: G:\My Drive\Code\Kallipos\book\data
-rw     414 B      students.dat
-rw     220 B      studentsd.dat
-rw     75 KB      sunset.jpg
-rw     75 KB      sunsetcopy.jpg
-rw     75 KB      sunsetcopyf.jpg
-rw     199 B      cars.dat
-rw     122 B      doubledata.dat
-rw     103 B      ChemicalElements.dat

   Directory: G:\My Drive\Code\Kallipos\book\data\test
-rw      72 B      Calendar.txt
-rw     16 KB     Letter.docx
hrw       0 B      demohidden.txt
```

Στο υπόδειγμα αυτό η dirRecurse κλήθηκε με παράμετρο το αντικείμενο File που αντιστοιχεί στον φάκελο G:\My Drive\Code\Kallipos\book\data. Ο φάκελος περιέχει τον υποφάκελο test. Οι πληροφορίες που αφορούν κάθε αρχείο έχουν ως εξής: Οι πρώτοι τρεις χαρακτήρες δηλώνουν κατά σειρά αν το αρχείο είναι κρυφό